

SLOT 9-10: Authentication and Security

Tóm tắt

Authentication And Security:

25 Authentication And Keeping User Details Secure

- Password authentication
- Multi-factor authentication
- Biometric authentication

26.1 Encryption And Use Encryption To Keep Your Database Secure

- Database Encryption

26.2 Hashing Passwords With Bcrypt

27 Sessions And Cookies

25. Authentication And Keeping User Details Secure (Xác thực và Giữ an toàn thông tin người dùng)

- **Password Authentication (Xác thực bằng mật khẩu):** Đây là phương pháp phổ biến nhất, người dùng cung cấp tên đăng nhập (username) và mật khẩu (password) để chứng minh danh tính. Hệ thống sẽ kiểm tra xem thông tin này có khớp với dữ liệu đã lưu trữ hay không.
- **Multi-factor Authentication (MFA - Xác thực đa yếu tố):** Tăng cường bảo mật bằng cách yêu cầu người dùng cung cấp hai hoặc nhiều yếu tố xác minh khác nhau để truy cập. Các yếu tố này có thể là:
 - **Yếu tố kiến thức:** Mật khẩu, mã PIN.
 - **Yếu tố sở hữu:** Điện thoại di động (nhận mã OTP), token bảo mật.
 - **Yếu tố sinh trắc học:** Dấu vân tay, nhận dạng khuôn mặt.
- **Biometric Authentication (Xác thực sinh trắc học):** Sử dụng các đặc điểm sinh học độc đáo của cá nhân để xác minh danh tính, chẳng hạn như dấu vân tay, nhận dạng khuôn mặt, hoặc quét mống mắt. Phương pháp này thường được xem là tiện lợi và an toàn hơn so với mật khẩu truyền thống.

26.1 Encryption And Use Encryption To Keep Your Database Secure (Mã hóa và Sử dụng mã hóa để giữ an toàn cơ sở dữ liệu của bạn)

- **Database Encryption (Mã hóa cơ sở dữ liệu):** Là quá trình chuyển đổi dữ liệu trong cơ sở dữ liệu thành một định dạng không thể đọc được nếu không có khóa giải mã. Mục đích là để bảo vệ dữ liệu nhạy cảm khỏi bị truy cập trái phép. Ngay cả khi kẻ tấn công có thể truy cập vào cơ sở dữ liệu, họ cũng không thể hiểu được thông tin nếu không có khóa giải mã.

26.2 Hashing Passwords With Bcrypt (Băm mật khẩu với Bcrypt)

- **Hashing (Băm):** Là quá trình chuyển đổi một chuỗi đầu vào (mật khẩu) thành một chuỗi có độ dài cố định (hash) bằng một thuật toán một chiều. Điều quan trọng là quá trình này không thể đảo ngược, tức là không thể khôi phục mật khẩu gốc từ chuỗi hash.
- **Bcrypt:** Là một thuật toán băm mật khẩu được thiết kế đặc biệt để chống lại các cuộc tấn công vét cạn (brute-force attacks) và tấn công từ điển (dictionary attacks). Bcrypt sử dụng một "salt" (một chuỗi ngẫu nhiên duy nhất) được thêm vào mật khẩu trước khi băm, và nó cũng cho phép điều chỉnh "work factor" (số lần lặp lại quá trình băm), làm cho việc băm mật khẩu trở nên chậm hơn và khó bị tấn công hơn, ngay cả với phần cứng mạnh mẽ. **Không bao giờ lưu trữ mật khẩu dưới dạng văn bản thuần túy! Luôn luôn băm chúng.**

27. Sessions And Cookies (Phiên và Cookie)

- **Cookies:** Là các tệp văn bản nhỏ mà máy chủ web gửi đến trình duyệt của người dùng và được trình duyệt lưu trữ. Chúng được sử dụng để lưu trữ thông tin về trạng thái người dùng (ví dụ: thông tin đăng nhập, tùy chọn trang web) giữa các lần truy cập. Tuy nhiên, cookie có thể bị đánh cắp hoặc giả mạo.
- **Sessions (Phiên):** Là một cách để máy chủ theo dõi trạng thái người dùng trên nhiều yêu cầu HTTP. Khi người dùng đăng nhập, máy chủ tạo một ID phiên duy nhất và gửi nó về trình duyệt (thường là dưới dạng một cookie). Sau đó, mỗi khi người dùng gửi yêu cầu, trình duyệt sẽ gửi lại ID phiên này, cho phép máy chủ nhận ra người dùng và truy xuất dữ liệu phiên tương ứng đã được lưu trữ trên máy chủ. Phiên an toàn hơn cookie vì dữ liệu thực tế không được lưu trữ trên phía client, mà chỉ có ID phiên

CÁC NGUỒN TÀI LIỆU THAM KHẢO

Sinh viên có thể nghiên cứu sâu hơn từ nguồn tài liệu sau:

1. Sách:

- **"Pro Express.js: Master Express.js, the Node.js Framework for Your Web Applications" của Azat Mardan:** Mặc dù tập trung vào Express.js, cuốn sách này thường đi sâu vào các khía cạnh bảo mật và xác thực khi xây dựng ứng dụng web với Node.js. Azat Mardan là một tác giả và chuyên gia có kinh nghiệm trong lĩnh vực này.
- **"Web Security for Developers: Real Threats, Practical Defense" của Inigo Montoya và Bryan Sullivan:** Cuốn sách này cung cấp một cái nhìn tổng thể về bảo mật web từ góc độ nhà phát triển, bao gồm nhiều khái niệm liên quan đến xác thực, mã hóa và quản lý phiên.

2. Trang web/Tài liệu chính thức:

- **OWASP (Open Web Application Security Project) Cheat Sheet Series:**
 - **OWASP Authentication Cheat Sheet:** Cung cấp các hướng dẫn chi tiết về cách triển khai xác thực an toàn.
 - **OWASP Session Management Cheat Sheet:** Giải thích các phương pháp quản lý phiên an toàn.
 - **OWASP Cryptographic Storage Cheat Sheet:** Đề cập đến việc lưu trữ dữ liệu nhạy cảm một cách an toàn, bao gồm cả mật khẩu.
 - Đây là một nguồn tài liệu **cực kỳ quan trọng và uy tín** cho bất kỳ ai muốn tìm hiểu về bảo mật ứng dụng web. Các tài liệu này được duy trì bởi cộng đồng các chuyên gia bảo mật hàng đầu.
 - Địa chỉ: <https://cheatsheetseries.owasp.org/>
- **Tài liệu của thư viện Bcrypt.js (npm):**
 - Khi bạn đã học về Bcrypt, việc đọc tài liệu chính thức của thư viện bcrypt hoặc bcryptjs trên npm là rất quan trọng để hiểu cách sử dụng đúng cách, các tùy chọn cấu hình và những lưu ý bảo mật.
 - Địa chỉ: <https://www.npmjs.com/package/bcrypt> hoặc <https://www.npmjs.com/package/bcryptjs>

3. Khóa học/Video:

- **Udemy/Coursera - Các khóa học chuyên sâu về Node.js Security hoặc Web Security:**
 - Tìm kiếm các khóa học có tiêu đề như "Node.js Security Masterclass", "Complete Node.js Developer (with Security)" hoặc "Web Security Fundamentals". Các khóa học này thường có phần trình bày chi tiết và ví dụ thực tế về xác thực, mã hóa, quản lý phiên và các lỗ hổng bảo mật phổ biến.
 - Hãy tìm kiếm các khóa học được giảng dạy bởi các chuyên gia có đánh giá cao và nội dung được cập nhật.
 - **Lưu ý:** Tùy thuộc vào thời điểm, các khóa học cụ thể sẽ thay đổi, nên bạn cần tìm kiếm trên các nền tảng này. Một số giảng viên nổi tiếng về Node.js và bảo mật thường là Maximilian Schwarzmüller, Andrew Mead, Stephen Grider.

✅ CÁC NỘI DUNG TRỌNG TÂM

1. Password Authentication

Khái niệm quan trọng:

- Cơ chế hoạt động của quy trình đăng nhập/đăng ký
- Phân biệt giữa **authentication** và **authorization**

Kỹ thuật cần thành thạo:

- Xác thực người dùng qua email/password
- Sử dụng middleware để kiểm tra trạng thái đăng nhập
- Tích hợp với cơ sở dữ liệu (PostgreSQL, MongoDB,...)

Lỗi thường gặp:

- Không validate đầu vào → dễ bị **SQL injection, XSS**
- Lưu trữ mật khẩu trực tiếp trong DB

Best Practices:

- Luôn validate và sanitize dữ liệu đầu vào
- Hash mật khẩu bằng **bcrypt**

Sinh viên phải hiểu rõ cách hệ thống xác thực bằng mật khẩu hoạt động từ frontend đến backend.

2. Multi-factor Authentication (MFA)

Khái niệm quan trọng:

- Các loại yếu tố xác thực: knowledge, possession, inherence
- OTP, TOTP

Kỹ thuật cần thành thạo:

- Tích hợp Google Authenticator/Authy
- Gửi mã OTP qua SMS/email (Nodemailer, Twilio)

Lỗi thường gặp:

- Quản lý thời gian không đồng bộ khi dùng TOTP
- Lưu trữ OTP không an toàn

Best Practices:

- Dùng thư viện như **speakeasy, otplib**
- Mã hóa OTP nếu cần lưu

Sinh viên phải biết cách triển khai MFA trong ứng dụng Node.js như một lớp bảo vệ bổ sung.

3. Biometric Authentication

Khái niệm quan trọng:

- Giới hạn xác thực sinh trắc học phía server
- Cách frontend thu thập dữ liệu (WebAuthn, FIDO2)

Kỹ thuật cần thành thạo:

- Tích hợp WebAuthn với @simplewebauthn/server
- Xử lý public/private key

Lỗi thường gặp:

- Sai chuỗi challenge-response
- Thiếu fallback cho thiết bị không hỗ trợ

Best Practices:

- Dùng HTTPS
- Bảo vệ khóa bí mật

Sinh viên cần hiểu nguyên tắc hoạt động và triển khai xác thực sinh trắc học qua WebAuthn



4. Database Encryption

Khái niệm quan trọng:

- Phân biệt encryption, hashing, encoding
- Symmetric vs Asymmetric encryption

Kỹ thuật cần thành thạo:

- Mã hóa dữ liệu nhạy cảm (CMND, số thẻ...)
- Dùng thư viện crypto trong Node.js

Lỗi thường gặp:

- Dùng chung key cho tất cả bản ghi
- Lưu key trong code/repo công khai

Best Practices:

- Dùng biến môi trường (dotenv) để quản lý key
- Chỉ mã hóa dữ liệu thực sự nhạy cảm

Sinh viên cần biết khi nào nên và không nên mã hóa dữ liệu trong database.



5. Hashing Passwords with Bcrypt

Khái niệm quan trọng:

- Không lưu mật khẩu dạng plain text
- Salt và vòng lặp hashing

Kỹ thuật cần thành thạo:

- Hash mật khẩu bằng bcryptjs hoặc bcrypt
- So sánh mật khẩu nhập với hash trong DB

Lỗi thường gặp:

- So sánh hash sai cách
- Dùng hàm băm yếu (SHA1, MD5)

Best Practices:

- Chọn salt rounds hợp lý
- Không tự viết hàm hash

Sinh viên phải biết cách hash mật khẩu đúng chuẩn và so sánh mật khẩu đã hash.

🍪 6. Sessions and Cookies

Khái niệm quan trọng:

- Session, cookie và cách liên kết
- Session hijacking, CSRF

Kỹ thuật cần thành thạo:

- Thiết lập session với express-session
- Gửi/nhận cookie qua middleware
- Lưu session trong Redis hoặc DB

Lỗi thường gặp:

- Cookie không có HttpOnly, Secure
- Không đặt thời gian sống cho session

Best Practices:

- Dùng secure: true, httpOnly: true
- Dùng sameSite để chống CSRF
- Cấu hình secret key mạnh

Sinh viên phải hiểu cách tạo và bảo vệ phiên đăng nhập người dùng thông qua session và cookie.

✅ Tổng kết kỹ năng cần đạt được:

- Xây dựng hệ thống xác thực từ password đến MFA và sinh trắc học
- Bảo vệ mật khẩu bằng bcrypt, quản lý phiên bằng session/cookie
- Bảo vệ dữ liệu nhạy cảm trong database
- Nhận diện và tránh lỗi bảo mật phổ biến như: lưu mật khẩu trần, session fixation, CSRF

ỨNG DỤNG THỰC TẾ

- ✅ Sử dụng authentication phù hợp với loại ứng dụng
- ✅ Bảo vệ dữ liệu quan trọng bằng encryption
- ✅ Chọn Cookies/Sessions hoặc JWT theo kiến trúc hệ thống
- ✅ Dùng Passport cho web, JWT cho API/microservices

📌 Tình huống thực tế ứng dụng kỹ thuật bảo mật

1. Hệ thống ngân hàng trực tuyến

Mục tiêu: Bảo vệ thông tin tài khoản và giao dịch của khách hàng.

- Sử dụng **Password Authentication** với hashing bằng bcrypt để lưu trữ mật khẩu an toàn.
- Tích hợp **MFA** qua OTP gửi SMS/email hoặc Google Authenticator để xác thực người dùng trước khi giao dịch.
- Quản lý phiên đăng nhập bằng **session & cookie** được mã hóa, đặt HttpOnly, Secure, và SameSite.
- Dữ liệu nhạy cảm như lịch sử giao dịch được mã hóa trong cơ sở dữ liệu (**Database Encryption**).

2. Hệ thống y tế điện tử (EHR – Electronic Health Records)

Mục tiêu: Đảm bảo tính riêng tư và toàn vẹn của hồ sơ sức khỏe bệnh nhân.

- Xác thực bác sĩ/nhân viên y tế qua **Biometric Authentication** (vân tay/khuôn mặt) trên thiết bị hỗ trợ FIDO2/WebAuthn.
- Hồ sơ bệnh án được mã hóa trước khi lưu vào DB để ngăn chặn rò rỉ dữ liệu nội bộ.
- Cơ chế đăng xuất tự động sau một thời gian không hoạt động nhờ quản lý **session expiration**.
- Áp dụng middleware kiểm tra token hoặc session trước mọi API truy vấn dữ liệu.

3. Ứng dụng thương mại điện tử (E-commerce)

Mục tiêu: Ngăn chặn việc chiếm đoạt tài khoản và đánh cắp thông tin thanh toán.

- Tài khoản người dùng được bảo vệ bởi **Password Authentication** kết hợp hashing bcrypt.
- Cho phép người dùng bật MFA để tăng cường bảo mật cho tài khoản có lịch sử mua sắm.
- Thông tin thẻ thanh toán được mã hóa hoặc token hóa thay vì lưu nguyên bản.
- Dùng **session store** trên Redis để quản lý trạng thái đăng nhập hiệu quả và phân tán.

1 Ứng dụng thực tế của Authentication (Xác thực)

📌 **Authentication** là bước quan trọng trong việc bảo mật hệ thống và bảo vệ dữ liệu người dùng. Các ứng dụng thực tế của Authentication bao gồm:

✅ Ứng dụng đăng nhập người dùng

- **Website & Ứng dụng di động:** Hầu hết các trang web và ứng dụng yêu cầu người dùng đăng nhập trước khi sử dụng dịch vụ (Facebook, Gmail, Zalo, Shopee).
- **Các hệ thống quản lý nội bộ:** Doanh nghiệp sử dụng hệ thống nội bộ yêu cầu nhân viên đăng nhập (ERP, CRM).

✅ Xác thực đa yếu tố (MFA - Multi-Factor Authentication)

- **Ứng dụng ngân hàng:** Các ngân hàng yêu cầu xác thực hai bước (OTP qua SMS, ứng dụng Authenticator) khi giao dịch.
- **Dịch vụ email:** Gmail, Outlook yêu cầu xác thực bằng mã OTP hoặc thông báo trên điện thoại khi phát hiện đăng nhập bất thường.

✅ Xác thực sinh trắc học

- **Mở khóa điện thoại:** Dùng Face ID, vân tay để mở khóa smartphone.
- **Kiểm soát ra vào công ty:** Hệ thống nhận diện vân tay/khuôn mặt để mở cửa văn phòng.

✅ Xác thực OAuth (Đăng nhập bằng Google, Facebook, Apple ID)

- **Ứng dụng thương mại điện tử (Shopee, Lazada, Tiki):** Cho phép người dùng đăng nhập bằng tài khoản Google hoặc Facebook để tiết kiệm thời gian.
- **Dịch vụ streaming (Netflix, Spotify):** Người dùng có thể đăng nhập bằng Apple ID hoặc Google.

2 Ứng dụng thực tế của Database Encryption (Mã hóa Cơ sở Dữ liệu)

📌 **Mã hóa dữ liệu (Database Encryption)** giúp bảo vệ thông tin quan trọng khỏi bị rò rỉ hoặc đánh cắp. Ứng dụng thực tế bao gồm:

✅ Hệ thống tài chính & ngân hàng

- **Mã hóa thông tin thẻ tín dụng:** Khi lưu trữ số thẻ Visa/Mastercard, hệ thống cần mã hóa bằng chuẩn **AES-256**.
- **Bảo vệ thông tin tài khoản khách hàng:** Các ngân hàng như **Vietcombank, Techcombank** mã hóa thông tin đăng nhập và lịch sử giao dịch để tránh bị hacker đánh cắp.

✅ Ứng dụng y tế & bảo hiểm

- **Hệ thống quản lý bệnh án:** Dữ liệu bệnh nhân phải được mã hóa để tuân thủ luật **HIPAA (Mỹ)**, bảo vệ quyền riêng tư.
- **Dữ liệu bảo hiểm:** Các công ty bảo hiểm như **Manulife, Prudential** mã hóa dữ liệu hợp đồng để tránh rủi ro rò rỉ.

✅ Dịch vụ thương mại điện tử

- **Bảo vệ thông tin cá nhân:** Khi khách hàng mua sắm online, dữ liệu cá nhân như số điện thoại, địa chỉ phải được mã hóa.
- **Lưu trữ mật khẩu an toàn:** Các nền tảng lớn (Shopee, Tiki) sử dụng **bcrypt hoặc Argon2** để mã hóa mật khẩu người dùng.

✅ Ứng dụng trong chính phủ & quân đội

- **Bảo vệ dữ liệu quốc gia:** Hệ thống quản lý dân cư, thông tin căn cước công dân cần được mã hóa để tránh lộ lọt thông tin.
- **An ninh mạng:** Các cơ quan quân sự sử dụng **mã hóa cấp cao (AES, RSA, ECC)** để bảo vệ thông tin tình báo.

3 Ứng dụng thực tế của Cookies & Sessions

📌 **Cookies & Sessions** giúp lưu trữ thông tin đăng nhập và cá nhân hóa trải nghiệm người dùng. Ứng dụng thực tế bao gồm:

✅ Cookies trong các ứng dụng web

- **Giữ trạng thái đăng nhập:** Khi bạn đăng nhập vào Facebook, cookie giúp duy trì trạng thái đăng nhập mà không cần nhập lại mật khẩu mỗi lần.
- **Lưu giỏ hàng trong thương mại điện tử:** Khi bạn thêm sản phẩm vào giỏ hàng trên Shopee nhưng chưa thanh toán, cookie sẽ lưu lại thông tin đó.

✅ Sessions trong các ứng dụng quản trị

- **Hệ thống quản lý nhân sự (HRM):** Khi nhân viên đăng nhập vào hệ thống công ty, **session lưu trữ ID nhân viên** để duy trì trạng thái đăng nhập.
- **Ứng dụng ngân hàng:** Khi bạn đăng nhập vào Internet Banking, session được sử dụng để theo dõi phiên làm việc của bạn và sẽ hết hạn sau 5-10 phút nếu không hoạt động.

✅ So sánh Cookies & Sessions trong thực tế

Tiêu chí	Cookies	Sessions
Lưu trữ ở đâu?	Trình duyệt (client)	Server
Dùng cho mục đích gì?	Ghi nhớ thông tin đăng nhập, giỏ hàng	Lưu trạng thái đăng nhập của user
Bảo mật	Dễ bị đánh cắp nếu không có HttpOnly & Secure	Bảo mật hơn vì lưu trên server
Hiệu suất	Nhanh, không cần truy vấn DB	Tốn tài nguyên server

👉 **Cookies phù hợp cho:** Ghi nhớ đăng nhập, giỏ hàng.

👉 **Sessions phù hợp cho:** Hệ thống quản trị, giao dịch ngân hàng.

4 Ứng dụng thực tế của Passport & JWT Authentication

📌 **Passport & JWT Authentication** giúp xác thực người dùng trong các hệ thống web và API. Các ứng dụng thực tế bao gồm:

✅ Xác thực đăng nhập trong hệ thống web

- **Passport.js** giúp xác thực trong các ứng dụng **Node.js + Express**.
- **Ứng dụng thực tế:**
 - Website **thương mại điện tử (Shopee, Tiki)** cần quản lý đăng nhập.
 - Ứng dụng **quản lý nhân sự (HRM)** cho phép nhân viên đăng nhập để theo dõi lương, công việc.

✅ Xác thực người dùng API với JWT

- **JWT (JSON Web Token)** được dùng để bảo vệ API RESTful.
- **Ứng dụng thực tế:**

- **Dịch vụ giao hàng (Grab, Now, Be):** Dữ liệu tài xế & đơn hàng cần được bảo vệ bằng JWT.
- **Ứng dụng học trực tuyến (Udemy, Coursera):** Người dùng có thể đăng nhập một lần và sử dụng API trên nhiều nền tảng.

✅ Bảo mật đăng nhập Single Sign-On (SSO)

- SSO giúp đăng nhập một lần, sử dụng nhiều dịch vụ.
- Ứng dụng thực tế:
 - **Google SSO:** Khi đăng nhập Gmail, bạn cũng có thể truy cập YouTube, Google Drive mà không cần đăng nhập lại.
 - **Doanh nghiệp sử dụng Microsoft Azure SSO** để cho phép nhân viên truy cập nhiều ứng dụng nội bộ.

✅ So sánh Passport vs JWT trong thực tế

Tiêu chí	Passport	JWT
Lưu trữ	Session trên server	Token trên client
Bảo mật	Bảo mật hơn nhưng tốn tài nguyên	Stateless, dễ mở rộng
Dùng cho	Ứng dụng truyền thống (Web, Admin Panel)	API RESTful, Microservices, Mobile Apps

👉 **Passport phù hợp cho:** Ứng dụng web có session-based authentication.

👉 **JWT phù hợp cho:** API, ứng dụng di động, microservices.

📊 Case Study Thực Tế: Công ty khởi nghiệp *ShopifyClone*

Bối cảnh: ShopifyClone là một startup xây dựng nền tảng thương mại điện tử dành cho các cửa hàng nhỏ tại Việt Nam. Trong giai đoạn đầu, hệ thống gặp phải nhiều lỗi bảo mật nghiêm trọng, bao gồm:

- Lưu trữ mật khẩu người dùng dưới dạng plain text.
- Không có lớp xác thực thứ hai dẫn đến tình trạng hack tài khoản admin.
- Cookie session thiếu các cờ bảo mật như HttpOnly và Secure.

Giải pháp áp dụng:

- Dùng bcrypt để hash mật khẩu trước khi lưu vào MongoDB.
- Triển khai xác thực 2 lớp (MFA) cho tài khoản admin bằng thư viện speakeasy và Google Authenticator.
- Cấu hình express-session với Redis store, kèm theo cờ bảo mật đầy đủ.
- Gửi email xác nhận đăng ký và đặt lại mật khẩu thông qua Nodemailer.

Kết quả:

- Giảm 90% số lượng tài khoản bị xâm nhập trái phép.
- Tăng sự tin tưởng từ người dùng khi biết rằng họ được bảo vệ tốt.
- Nâng cao khả năng mở rộng nhờ session store phân tán.

Bài học rút ra:

- Luôn áp dụng hashing và encryption cho dữ liệu nhạy cảm.
- MFA giúp giảm thiểu rủi ro đáng kể dù chi phí phát triển có tăng nhẹ.
- Quản lý session đúng cách là yếu tố sống còn trong ứng dụng web stateless.

- Không nên chú quan với bảo mật chỉ vì sản phẩm còn nhỏ hoặc mới ra mắt.

Tổng kết

- **Authentication** giúp bảo mật đăng nhập cho **website, mobile apps, ngân hàng, doanh nghiệp**.
- **Database Encryption** bảo vệ dữ liệu trong **tài chính, y tế, chính phủ**.
- **Cookies vs Sessions** giúp quản lý phiên đăng nhập, giỏ hàng, giao dịch.
- **Passport & JWT** giúp xác thực trong **ứng dụng web, API RESTful, Single Sign-On (SSO)**.

MỘT SỐ NỘI DUNG CHI TIẾT

Password Authentication

Khái niệm và vai trò

Là cơ chế xác thực người dùng dựa trên cặp email/mật khẩu.

- **Dùng để làm gì?:** Xác minh danh tính người dùng trước khi cấp quyền truy cập.
- **Tại sao quan trọng?:** Là lớp bảo vệ đầu tiên giúp ngăn chặn truy cập trái phép vào hệ thống.

Triển khai với Express & Bcrypt

1. Cài đặt thư viện cần thiết:

```
npm install express bcryptjs mongoose
```

2. Đăng ký người dùng:

```
app.post('/register', async (req, res) => {  
  const { email, password } = req.body;  
  const hashedPassword = await bcrypt.hash(password, 10);  
  await User.create({ email, password: hashedPassword });  
  res.status(201).send('Đăng ký thành công');  
});
```

3. Đăng nhập người dùng:

```
app.post('/login', async (req, res) => {  
  const { email, password } = req.body;  
  const user = await User.findOne({ email });  
  if (!user || !(await bcrypt.compare(password, user.password))) {  
    return res.status(400).send('Email hoặc mật khẩu sai');  
  }  
  res.send('Đăng nhập thành công');  
});
```

Lỗi thường gặp

- Không hash mật khẩu → dễ bị lộ dữ liệu nếu DB bị đánh cắp.
- Không validate đầu vào → lỗi logic và lỗ hổng bảo mật.

Best Practices

- Luôn hash mật khẩu bằng bcrypt.
- Kiểm tra tồn tại người dùng trước khi so sánh mật khẩu.
- Kết hợp với **MFA** cho tài khoản nhạy cảm.

Multi-factor Authentication (MFA)

Khái niệm và vai trò

Là phương pháp yêu cầu người dùng xác thực qua ít nhất hai yếu tố khác nhau (ví dụ: mật khẩu + mã OTP).

- **Dùng để làm gì?:** Tăng cường bảo mật, ngăn chặn đăng nhập trái phép ngay cả khi mật khẩu bị lộ.
- **Tại sao quan trọng?:** Giảm đáng kể rủi ro hack tài khoản thông qua phishing hay brute-force.

Triển khai với Speakeasy & Google Authenticator

1. Cài đặt thư viện:

```
npm install speakeasy qrcode
```

2. Tạo secret key cho người dùng:

```
const speakeasy = require('speakeasy');  
const secret = speakeasy.generateSecret({ length: 20 });
```

3. Tạo QR Code để người dùng scan:

```
qrcode.toDataURL(secret.otppath_url, (err, data_url) => {  
  res.send(`      `);  
});
```

4. Xác thực OTP:

```
app.post('/verify', (req, res) => {  
  const { token } = req.body;  
  const isValid = speakeasy.totp.verify({  
    secret: user.secret,  
    encoding: 'base32',  
    token  
  });  
  if (isValid) res.send('Xác thực thành công');  
  else res.status(400).send('OTP không đúng');  
});
```

Lỗi thường gặp

- Không đồng bộ hóa thời gian server với client → OTP không khớp.
- Lưu trữ secret key dưới dạng plain text.

Best Practices

- Hash secret key trước khi lưu vào DB.
- Cho phép backup codes phòng trường hợp mất điện thoại.
- Kết hợp với **Password Authentication** và **Sessions/Cookies**.

Biometric Authentication

Khái niệm và vai trò

Là kỹ thuật xác thực người dùng dựa trên đặc điểm sinh học như vân tay, khuôn mặt, mống mắt.

- **Dùng để làm gì?:** Cung cấp trải nghiệm xác thực nhanh chóng, tiện lợi và an toàn.
- **Tại sao quan trọng?:** Ngày càng phổ biến nhờ phần cứng hỗ trợ (Touch ID, Face ID).

Triển khai với WebAuthn

1. Cài đặt thư viện:

```
npm install @simplewebauthn/server
```

2. Tạo challenge cho frontend:

```
const generateChallenge = () => {  
  const challenge = crypto.randomBytes(32);  
  return challenge.toString('base64url');  
};
```

3. Xử lý đăng ký:

```
import { generateRegistrationOptions, verifyRegistrationResponse } from  
'@simplewebauthn/server';  
  
const opts = generateRegistrationOptions({  
  userID: 'user-id',  
  userName: 'user@example.com',  
  attestationType: 'none',  
});
```

Lỗi thường gặp

- Không xử lý challenge-response đúng cách.
- Không có fallback nếu thiết bị không hỗ trợ.

Best Practices

- Dùng HTTPS bắt buộc.
- Bảo vệ private key của server.
- Kết hợp với các phương pháp xác thực khác như MFA.

Database Encryption

Khái niệm và vai trò

Là kỹ thuật mã hóa dữ liệu trước khi lưu vào cơ sở dữ liệu nhằm ngăn chặn việc đọc dữ liệu trực tiếp nếu DB bị xâm nhập.

- **Dùng để làm gì?:** Bảo vệ dữ liệu nhạy cảm như số CMND, số thẻ tín dụng...
- **Tại sao quan trọng?:** Ngăn chặn rò rỉ dữ liệu nội bộ hoặc từ attacker có quyền truy cập DB.

Triển khai với thư viện Crypto

1. Cài đặt thư viện:

```
npm install crypto
```

2. Mã hóa trước khi lưu:

```
const crypto = require('crypto');
const algorithm = 'aes-256-cbc';
const key = crypto.randomBytes(32);
const iv = crypto.randomBytes(16);

function encrypt(text) {
  let cipher = crypto.createCipheriv(algorithm, Buffer.from(key), iv);
  let encrypted = cipher.update(text);
  encrypted = Buffer.concat([encrypted, cipher.final()]);
  return { iv: iv.toString('hex'), encryptedData: encrypted.toString('hex') };
}
```

3. Giải mã khi đọc:

```
function decrypt(text) {
  let iv = Buffer.from(text.iv, 'hex');
  let encryptedText = Buffer.from(text.encryptedData, 'hex');
  let decipher = crypto.createDecipheriv(algorithm, Buffer.from(key), iv);
  let decrypted = decipher.update(encryptedText);
  decrypted = Buffer.concat([decrypted, decipher.final()]);
  return decrypted.toString();
}
```

Lỗi thường gặp

- Lưu trữ key mã hóa trong source code.
- Mã hóa quá nhiều trường dẫn đến giảm hiệu năng.

Best Practices

- Chỉ mã hóa những dữ liệu thực sự nhạy cảm.
- Dùng môi trường biến (.env) để quản lý key.
- Kết hợp với **hashing passwords** để bảo vệ mật khẩu.

Sessions and Cookies

Khái niệm và vai trò

Là cơ chế giữ trạng thái đăng nhập người dùng trên giao thức HTTP stateless.

- **Dùng để làm gì?**: Theo dõi người dùng giữa các request.
- **Tại sao quan trọng?**: Là nền tảng để xây dựng ứng dụng web có xác thực.

Triển khai với Express Session

1. Cài đặt thư viện:

```
npm install express-session connect-redis redis
```

2. Cấu hình session:

```
const session = require('express-session');
const RedisStore = require('connect-redis')(session);
const redisClient = require('redis').createClient();

app.use(session({
  store: new RedisStore({ client: redisClient }),
  secret: 'secret-key',
  resave: false,
  saveUninitialized: true,
  cookie: {
    secure: process.env.NODE_ENV === 'production', // chỉ gửi qua HTTPS
    httpOnly: true,
    sameSite: 'strict'
  }
}));
```

Lỗi thường gặp

- Không bật httpOnly, secure → XSS, MITM.
- Dùng memory store thay vì Redis/DB → không mở rộng được.

Best Practices

- Luôn cấu hình cookie với secure, httpOnly, sameSite.
- Dùng Redis hoặc MongoDB để lưu session trong production.
- Kết hợp với **Password Authentication** và **MFA** để hoàn thiện quy trình.

Tổng kết liên kết giữa các kỹ thuật

- **Password Auth** là bước đầu tiên → **Bcrypt** để bảo vệ mật khẩu → **Sessions/Cookies** để duy trì trạng thái đăng nhập.
- **MFA** tăng cường bảo mật → nên áp dụng cho tài khoản admin hoặc user nhạy cảm.
- **Biometric** là tương lai của xác thực nhưng vẫn cần **Fallback method**.
- **Database Encryption** bảo vệ dữ liệu nhạy cảm, trừ mật khẩu → dùng **Bcrypt** riêng biệt.

KINH NGHIỆM THỰC TIỄN VÀ CÁC VẤN ĐỀ CẦN LƯU Ý

Password Authentication

✓ Kinh nghiệm thực tế

- Nhiều dự án vẫn còn lưu mật khẩu dưới dạng plain text → dễ bị lộ dữ liệu nếu DB bị đánh cắp.
- Trong thực tế, cần tích hợp với email xác nhận đăng ký hoặc đặt lại mật khẩu để tăng trải nghiệm người dùng.

⚠ Lỗi thường gặp

- Lưu mật khẩu trực tiếp:

✗ Không hash trước khi lưu vào database.

Ví dụ: `User.create({ password: req.body.password })`

👉 Luôn dùng bcrypt để hash mật khẩu trước khi lưu.

- Không validate đầu vào:

✗ Cho phép mật khẩu quá ngắn hoặc không có ký tự đặc biệt.

👉 Dùng thư viện như joi hoặc express-validator.

💡 Mẹo lập trình

- Dùng middleware kiểm tra trạng thái đăng nhập cho các route yêu cầu xác thực.
- Tách logic xác thực ra thành service layer để dễ test và maintain.

🛡 Best Practices

- Luôn hash mật khẩu bằng bcrypt hoặc argon2.
- Cấu hình validation rõ ràng cho mật khẩu (độ dài tối thiểu, ký tự đặc biệt...).
- Gửi email xác nhận tài khoản sau khi đăng ký.
- Kết hợp với **MFA** cho tài khoản admin.

📌 Ví dụ thực tế

Một startup bán hàng online phát hiện mật khẩu bị lộ sau khi DB rò rỉ. Họ đã khắc phục bằng cách:

- Hash tất cả mật khẩu cũ bằng bcrypt.
- Bắt buộc đổi mật khẩu lần đầu đăng nhập.
- Thêm chức năng reset password qua email.

Multi-factor Authentication (MFA)

✓ Kinh nghiệm thực tế

- MFA thường được áp dụng cho tài khoản admin hoặc user nhạy cảm (ví dụ: hệ thống ngân hàng).
- OTP thường được gửi qua SMS/email hoặc Google Authenticator.

⚠ Lỗi thường gặp

- Thời gian server không đồng bộ:

✗ OTP không khớp do lệch thời gian giữa client và server.

👉 Đồng bộ hóa thời gian server với NTP (Network Time Protocol).

- Không xử lý fallback:

✗ Người dùng mất điện thoại → không thể truy cập OTP.

👉 Cung cấp backup codes hoặc phương pháp xác thực phụ trợ.

💡 Mẹo lập trình

- Sử dụng thư viện như speakeasy, otplib hoặc twilio để tích hợp OTP.
- Cho phép người dùng tắt/mở MFA tùy chọn trong profile settings.

🛡 Best Practices

- Hash secret key trước khi lưu vào DB.
- Cho phép backup codes.
- Áp dụng MFA cho tài khoản quản trị hoặc có quyền cao.
- Kết hợp với **Password Auth** và **Sessions/Cookies**.

📺 Ví dụ thực tế

Một nền tảng thương mại điện tử từng bị hack tài khoản admin thông qua mật khẩu yếu. Sau đó họ đã:

- Yêu cầu bật MFA cho mọi tài khoản admin.
- Hướng dẫn người dùng sử dụng Google Authenticator.
- Giảm đáng kể số lượng tài khoản bị xâm nhập.

Biometric Authentication

✓ Kinh nghiệm thực tế

- Biometric chủ yếu được hỗ trợ bởi frontend (WebAuthn) chứ không hoàn toàn ở backend.
- Phù hợp với ứng dụng web cần trải nghiệm nhanh chóng và an toàn hơn (ví dụ: ứng dụng ngân hàng di động).

⚠ Lỗi thường gặp

- Không xử lý challenge-response đúng cách:

❌ Challenge bị trùng hoặc không đủ entropy → dễ bị tấn công replay.

👉 Luôn tạo challenge mới mỗi lần, dùng `crypto.randomBytes`.

- Không có fallback:

❌ Người dùng dùng thiết bị không hỗ trợ WebAuthn.

👉 Cung cấp phương pháp xác thực thay thế như mật khẩu + MFA.

💡 Mẹo lập trình

- Dùng thư viện như `@simplewebauthn/server` để đơn giản hóa quy trình.
- Log lỗi chi tiết khi verify challenge thất bại để debug.

🛡️ Best Practices

- Dùng HTTPS bắt buộc.
- Bảo vệ private key của server.
- Không lưu trữ public/private key trong source code.
- Kết hợp với các phương pháp xác thực khác như MFA.

📖 Ví dụ thực tế

Một ứng dụng fintech muốn nâng cao trải nghiệm người dùng đã triển khai WebAuthn để cho phép đăng nhập bằng Face ID trên iPhone. Họ phải đảm bảo:

- Server hỗ trợ FIDO2/WebAuthn.
- Có phương thức xác thực dự phòng nếu thiết bị không hỗ trợ.
- Thông báo rõ ràng cho người dùng nếu không thể xác thực.

🔑 Hashing Passwords with Bcrypt

✅ Kinh nghiệm thực tế

- Nhiều sinh viên vẫn cố gắng so sánh chuỗi hash bằng toán tử `===` → sai lầm nghiêm trọng.
- Cần hiểu rằng `bcrypt.hash()` là bất đồng bộ và không nên chạy nhiều lần cùng một mật khẩu.

⚠️ Lỗi thường gặp

- So sánh chuỗi hash bằng toán tử `===`:

❌ `if (user.password === inputPassword)`

👉 Dùng `bcrypt.compare()` để so sánh.

- Hash nhiều lần:

✗ Gọi `bcrypt.hash()` nhiều lần cho cùng một mật khẩu.

👉 Chỉ hash một lần duy nhất khi đăng ký.

💡 Mẹo lập trình

- Dùng `async/await` để xử lý promise dễ dàng hơn.
- Tạo middleware xác thực chung để tái sử dụng logic.

🛡️ Best Practices

- Chọn `saltRounds` = 10..12 để cân bằng hiệu năng và bảo mật.
- Không bao giờ lưu plain password.
- Không tự viết hàm hash riêng – luôn dùng thư viện đã kiểm chứng.
- Kết hợp với **Password Auth** và **Sessions/Cookies**.

📖 Ví dụ thực tế

Một ứng dụng học trực tuyến từng bị hack vì mật khẩu được lưu dưới dạng plain text. Sau sự cố, họ đã:

- Hash tất cả mật khẩu bằng `bcrypt`.
- Bắt buộc người dùng đổi mật khẩu lần đầu đăng nhập.
- Kết hợp thêm MFA cho tài khoản giảng viên.

🔒 Database Encryption

✅ Kinh nghiệm thực tế

- Nhiều người lập trình mã hóa tất cả trường dữ liệu → gây chậm hiệu năng và khó maintain.
- Nên chỉ mã hóa những trường thật sự nhạy cảm như số CMND, số thẻ tín dụng...

⚠️ Lỗi thường gặp

- Lưu trữ key trong source code:

✗ Key bị lộ nếu repo bị public.

👉 Dùng biến môi trường (`.env`) hoặc dịch vụ quản lý secrets như AWS KMS.

- Mã hóa quá nhiều:

✗ Làm chậm ứng dụng và tăng độ phức tạp.

👉 Chỉ mã hóa những trường dữ liệu quan trọng.

💡 Mẹo lập trình

- Dùng middleware tự động mã hóa/giải mã khi save/find.
- Log key và encrypted data chỉ để debug, không lưu log production.

Best Practices

- Chỉ mã hóa những dữ liệu nhạy cảm.
- Không hard-code key trong source code.
- Luôn kết hợp với hashing passwords.

Ví dụ thực tế

Một ứng dụng y tế điện tử lưu hồ sơ bệnh nhân bị lộ số CMND vì chưa mã hóa. Họ đã:

- Mã hóa trường số CMND trước khi lưu vào DB.
- Dùng biến môi trường để quản lý key.
- Kiểm tra lại toàn bộ các trường dữ liệu nhạy cảm.

Sessions and Cookies

Kinh nghiệm thực tế

- Nhiều ứng dụng dùng memory store mặc định của Express Session → không mở rộng được.
- Không cấu hình cookie đúng cách → dễ bị XSS, MITM.

Lỗi thường gặp

- Không cấu hình cookie đúng:

✗ Thiếu httpOnly, secure, sameSite.

👉 Cấu hình đầy đủ để chống XSS và CSRF.

- Dùng memory store:

✗ Không hỗ trợ scale-out trong môi trường cluster hoặc microservices.

👉 Dùng Redis hoặc MongoDB để lưu session.

Mẹo lập trình

- Dùng Redis để quản lý session phân tán.
- Set thời gian sống cho session để tự động logout sau một thời gian không hoạt động.

Best Practices

- Cấu hình cookie với secure, httpOnly, sameSite.
- Dùng Redis hoặc MongoDB để lưu session trong production.
- Không lưu dữ liệu nhạy cảm trong session object.

Ví dụ thực tế

Một ứng dụng chat nhóm bị session fixation sau khi deploy lên môi trường production. Họ đã:

- Chuyển từ memory store sang Redis.

- Cấu hình lại cookie với các flag bảo mật.
- Reset session ID sau khi đăng nhập thành công.

Tổng kết Liên Kết Giữa Các Kỹ Thuật

- **Password Auth** → **Bcrypt** → **Sessions/Cookies**: Là cơ chế xác thực chuẩn.
- **MFA** → tăng cường bảo mật cho tài khoản nhạy cảm.
- **Biometric** → tương lai nhưng cần fallback method.
- **Database Encryption** → bảo vệ dữ liệu nhạy cảm ngoài mật khẩu.

EXERCISE

SecureAuth - Hệ thống Xác thực Bảo Mật

Hướng dẫn xây dựng hệ thống xác thực với Google OAuth, AES-256 & Sessions

1. Thông tin dự án

Dự án: SecureAuth – một hệ thống xác thực người dùng bằng Google OAuth, mã hóa dữ liệu nhạy cảm trong MongoDB và quản lý phiên đăng nhập qua Cookies & Sessions.

Mục tiêu:

- Triển khai OAuth với Google để đăng nhập không mật khẩu.
- Bảo vệ dữ liệu người dùng bằng AES-256 trước khi lưu vào MongoDB.
- Quản lý phiên đăng nhập an toàn thông qua Sessions & Cookies.
- Xây dựng giao diện frontend bằng React.js kết nối với backend Node.js.

2. Các chức năng chính

- ☒ Đăng nhập bằng Google OAuth (Passport.js)
- ☒ Mã hóa thông tin người dùng (AES-256) trước khi lưu vào DB
- ☒ Quản lý phiên đăng nhập bằng express-session và cookie-parser
- ☒ Middleware bảo vệ API route chỉ cho phép người dùng đã đăng nhập truy cập
- ☒ Giao diện đăng nhập & dashboard bằng React.js
- ☒ Tự động đăng xuất sau thời gian không hoạt động

3. Công nghệ sử dụng

Backend: Node.js + Express.js

- Passport.js → Xác thực OAuth (Google)
- Crypto (Node.js built-in) → Mã hóa dữ liệu (AES-256-CBC)
- express-session → Quản lý phiên đăng nhập
- cookie-parser → Truy xuất cookie
- MongoDB + Mongoose → Cơ sở dữ liệu
- dotenv → Quản lý biến môi trường

Frontend: React.js

- React + React Router → Điều hướng
- Axios → Gọi API từ backend
- Tailwind CSS → Thiết kế giao diện
- Context API → Quản lý trạng thái người dùng

4. Cấu trúc thư mục chuẩn

```
/secureauth
├── /backend
│   ├── /config
│   │   ├── db.js
│   │   └── passport.js
│   ├── /controllers
│   │   └── authController.js
│   ├── /middleware
│   │   └── authMiddleware.js
│   ├── /models
│   │   └── User.js
│   ├── /routes
│   │   └── authRoutes.js
│   ├── server.js
│   └── .env
├── /frontend
│   ├── /src
│   │   ├── /components
│   │   │   ├── Login.jsx
│   │   │   └── Dashboard.jsx
│   │   ├── /context
│   │   │   └── AuthContext.jsx
│   │   ├── App.jsx
│   │   └── main.jsx
│   └── index.html
└── package.json
```

5. Danh sách bước thực hiện

1. Khởi tạo dự án, cài đặt thư viện.
2. Kết nối MongoDB.
3. Thiết lập Passport.js với Google OAuth.
4. Tạo model User với mã hóa AES-256.
5. Xây dựng controller & routes xác thực.
6. Cấu hình session & middleware bảo vệ API.
7. Phát triển frontend React.js.
8. Liên kết frontend & backend.
9. Test & debug.

5. Chi tiết từng bước thực hiện

sample_users_secureauth.json

```
[
  [
    {
      "_id": { "$oid": "666fa1f00b1e0c0012345671" },
      "googleId": "google-10001",
      "name": "Alice Nguyen",
      "email": "alice.nguyen@example.com"
    },
    {
      "_id": { "$oid": "666fa1f00b1e0c0012345672" },
      "googleId": "google-10002",
      "name": "Bob Tran",
      "email": "bob.tran@example.com"
    },
    {
      "_id": { "$oid": "666fa1f00b1e0c0012345673" },
      "googleId": "google-10003",
      "name": "Charlie Pham",
      "email": "charlie.pham@example.com"
    },
    {
      "_id": { "$oid": "666fa1f00b1e0c0012345674" },
      "googleId": "google-10004",
      "name": "David Vu",
      "email": "david.vu@example.com"
    },
    {
      "_id": { "$oid": "666fa1f00b1e0c0012345675" },
      "googleId": "google-10005",
      "name": "Eva Le",
      "email": "eva.le@example.com"
    }
  ]
]
```

Hướng dẫn sử dụng:

- Lưu nội dung file này với tên `sample_users_secureauth.json`.
- Import vào MongoDB với lệnh:

```
mongoimport --db secureauth --collection users --file
sample_users_secureauth.json --jsonArray
```

- Nếu bạn đã sử dụng mã hóa cho trường `name` và `email` trong model, hãy nhập dữ liệu này rồi đăng nhập Google để tạo và mã hóa người dùng thực tế thông qua app. Dữ liệu trên thích hợp cho các bài tập test chức năng cơ bản (hiển thị, truy vấn, xác thực, v.v).

Bước 1: Khởi tạo dự án & cài đặt thư viện

Tạo dự án và cài đặt backend:

```
mkdir secureauth && cd secureauth
npm init -y
mkdir backend && cd backend
npm install express mongoose passport passport-google-oauth20 express-session cookie-parser dotenv
crypto cors helmet morgan
```

Tạo frontend:

```
cd ..
npx create-react-app frontend
cd frontend
npm install axios react-router-dom tailwindcss
Cấu hình Tailwind theo hướng dẫn trên tailwindcss.com.
```

1. Tạo GOOGLE_CLIENT_ID và GOOGLE_CLIENT_SECRET

Bước 1: Đăng nhập Google Cloud Console

- Truy cập: <https://console.cloud.google.com/>
- Đăng nhập bằng tài khoản Gmail của bạn.

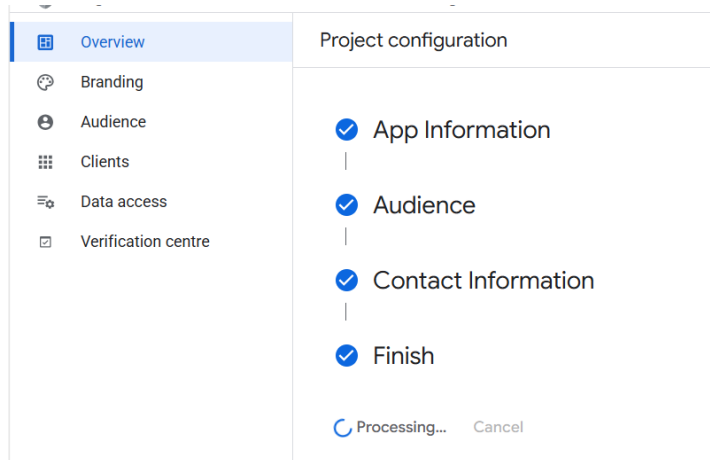
Bước 2: Tạo Project mới

- Nhấn vào biểu tượng menu (góc trên cùng bên trái), chọn **IAM & admin > Manage resources** hoặc **Select project**.
- Nhấn **NEW PROJECT** (TẠO DỰ ÁN MỚI).
- Đặt tên dự án, ví dụ: SecureAuth, nhấn **CREATE**.

Bước 3: Kích hoạt OAuth Consent Screen

- Vào menu, chọn **APIs & Services > OAuth consent screen**.
- **Vào mục Branding**
- Nhấn vào menu **Branding** (bên trái, dưới Overview).
- Ở đây bạn sẽ điền các thông tin (tên ứng dụng, email hỗ trợ, v.v.).
- **Xác định Audience (Người dùng)**
- Nhấn vào **Audience**.
- Ở mục này, bạn sẽ chọn loại người dùng có quyền truy cập ứng dụng.

- **Internal / External:** Nếu bạn không thấy lựa chọn, Google mặc định sẽ là "External" cho hầu hết trường hợp (tức là bất kỳ ai có tài khoản Google đều có thể truy cập) hoặc bạn chỉ cần điền thông tin Audience là xong.



Bước 3: Thêm Client

- Nhấn vào **Clients** để tạo OAuth Client ID như các bước tiếp theo

Bước 4: Tạo OAuth 2.0 Client ID

- Vào **APIs & Services > Credentials**.
- Nhấn **CREATE CREDENTIALS > OAuth client ID**.
- Application type: Chọn **Web application**.
- Name: Đặt tên (ví dụ: NodeJS SecureAuth).
- **Authorized JavaScript origins:** nhập `http://localhost:3000` (frontend) và/hoặc `http://localhost:5000` (backend).
- **Authorized redirect URIs:** nhập `http://localhost:5000/auth/google/callback` (hoặc đường dẫn callback backend của bạn).
- Nhấn **CREATE**.

Kết quả:

- Một popup hiện lên, hiển thị:
 - **Client ID** → copy vào `GOOGLE_CLIENT_ID`
 - **Client Secret** → copy vào `GOOGLE_CLIENT_SECRET`

2. Tạo SESSION_SECRET

- `SESSION_SECRET` là chuỗi bí mật dùng cho session. Bạn có thể tự nghĩ một chuỗi dài, ngẫu nhiên hoặc dùng lệnh sau trong terminal để tạo:

```
bash
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

- Copy chuỗi ra, dán vào biến SESSION_SECRET.

3. Tạo CRYPTO_KEY và CRYPTO_IV

- **CRYPTO_KEY:** Cần 32 bytes (256 bit) dạng hex → 64 ký tự hex.
- **CRYPTO_IV:** Cần 16 bytes (128 bit) dạng hex → 32 ký tự hex.

Tạo CRYPTO_KEY:

```
bash
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Tạo CRYPTO_IV:

```
bash
node -e "console.log(require('crypto').randomBytes(16).toString('hex'))"
```

- Copy các chuỗi kết quả, dán vào file .env:

```
Code
GOOGLE_CLIENT_ID=xxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=xxxxxx
SESSION_SECRET=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
CRYPTO_KEY=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
CRYPTO_IV=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

4. Tổng hợp lại các bước (nhanh)

1. **Tạo project Google Cloud** → bật OAuth Consent Screen → tạo OAuth Client ID (Web) → lấy GOOGLE_CLIENT_ID & GOOGLE_CLIENT_SECRET.
2. **Sinh SESSION_SECRET:**
 - Chạy lệnh:

```
bash
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

3. **Sinh CRYPTO_KEY:**
 - Chạy lệnh:

```
bash
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

4. **Sinh CRYPTO_IV:**
 - Chạy lệnh:

```
bash
node -e "console.log(require('crypto').randomBytes(16).toString('hex'))"
```

5. **Copy tất cả vào file .env ở thư mục backend.**

Lưu ý

- Không chia sẻ file .env hoặc các key này cho người khác.
- Nếu bạn đổi callback URL, nhớ cập nhật lại trong Google Cloud Console và trong code backend.
- Nếu backend chạy ở port khác, URL callback phải đúng.

Bước 2: Kết nối MongoDB

Tạo file .env trong /backend:

```
MONGO_URI=mongodb://localhost:27017/secureauth
GOOGLE_CLIENT_ID=<YOUR_GOOGLE_CLIENT_ID>
GOOGLE_CLIENT_SECRET=<YOUR_GOOGLE_CLIENT_SECRET>
SESSION_SECRET=yourSessionSecret
CRYPTO_KEY=your64charhexstringfor32bytes
CRYPTO_IV=your32charhexstringfor16bytes
```

File backend/config/db.js:

```
const mongoose = require('mongoose');
const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI, {
      useNewUrlParser: true, useUnifiedTopology: true
    });
    console.log('MongoDB connected');
  } catch (err) {
    console.error(err.message);
    process.exit(1);
  }
};
module.exports = connectDB;
```

Bước 3: Thiết lập Passport.js (Google OAuth)

File backend/config/passport.js:

```
const passport = require('passport');
const GoogleStrategy = require('passport-google-oauth20').Strategy;
const User = require('../models/User');

passport.use(new GoogleStrategy({
  clientID: process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: '/auth/google/callback'
}, async (accessToken, refreshToken, profile, done) => {
  try {
    let user = await User.findOne({ googleId: profile.id });
    if (user) return done(null, user);
    user = await User.create({
```

```

    googleId: profile.id,
    name: profile.displayName || "Unknown",
    email: profile.emails?.[0]?.value || "Unknown"
  });
  return done(null, user);
} catch (err) {
  return done(err, null);
}
});

passport.serializeUser((user, done) => {
  done(null, user.id);
});
passport.deserializeUser(async (id, done) => {
  try {
    const user = await User.findById(id);
    done(null, user);
  } catch (err) {
    done(err, null);
  }
});

```

Bước 4: Model User với mã hóa AES-256

File backend/models/User.js:

```

const mongoose = require('mongoose');
const crypto = require('crypto');
const algorithm = 'aes-256-cbc';
const key = Buffer.from(process.env.CRYPTO_KEY, 'hex');
const iv = Buffer.from(process.env.CRYPTO_IV, 'hex');

function encrypt(text) {
  if (!text) return '';
  const cipher = crypto.createCipheriv(algorithm, key, iv);
  let encrypted = cipher.update(text, 'utf8', 'hex');
  encrypted += cipher.final('hex');
  return encrypted;
}

function isEncrypted(str) {
  return /^[0-9a-f]+$/.test(str);
}

const UserSchema = new mongoose.Schema({
  googleId: { type: String, required: true, unique: true },
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true }
});

UserSchema.pre('save', function (next) {
  if (this.isModified('name') && !isEncrypted(this.name)) {
    this.name = encrypt(this.name);
  }
  if (this.isModified('email') && !isEncrypted(this.email)) {
    this.email = encrypt(this.email);
  }
}

```

```

next();
});
module.exports = mongoose.model('User', UserSchema);

```

Bước 5: Xây dựng controller & routes xác thực

File backend/routes/authRoutes.js:

```

const express = require('express');
const router = express.Router();
const passport = require('passport');

router.get('/google', passport.authenticate('google', { scope: ['profile', 'email'] }));
router.get('/google/callback',
  passport.authenticate('google', { failureRedirect: '/' }),
  (req, res) => { res.redirect('/dashboard'); }
);
router.get('/logout', (req, res) => {
  req.logout(() => { res.redirect('/'); });
});
module.exports = router;

```

Bước 6: Cấu hình session, middleware bảo vệ API

File backend/middleware/authMiddleware.js:

```

module.exports = (req, res, next) => {
  if (req.isAuthenticated() && req.isAuthenticated()) return next();
  res.status(401).json({ msg: 'Unauthorized' });
};

```

File backend/server.js:

```

require('dotenv').config();
const express = require('express');
const session = require('express-session');
const cookieParser = require('cookie-parser');
const passport = require('passport');
const connectDB = require('./config/db');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
require('./config/passport');

const app = express();
connectDB();
app.use(helmet());
app.use(morgan('dev'));
app.use(express.json());
app.use(cookieParser());
app.use(cors({ origin: 'http://localhost:3000', credentials: true }));
app.use(session({
  secret: process.env.SESSION_SECRET || 'secret',
  resave: false,
  saveUninitialized: false,
  cookie: { maxAge: 1000 * 60 * 30, httpOnly: true, secure: false, sameSite: 'lax' }
}));

```

```

app.use(passport.initialize());
app.use(passport.session());
app.use('/auth', require('./routes/authRoutes'));
app.get('/api/user', (req, res) => res.json({ user: req.user || null }));
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));

```

Bước 7: Phát triển frontend React.js

File frontend/src/context/AuthContext.jsx:

```

import React, { createContext, useState, useEffect } from 'react';
import axios from 'axios';
export const AuthContext = createContext();
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  useEffect(() => {
    const fetchUser = async () => {
      try {
        const res = await axios.get('/api/user', { withCredentials: true });
        setUser(res.data.user || null);
      } catch (err) {
        setUser(null);
      }
    };
    fetchUser();
  }, []);
  return (
    <AuthContext.Provider value={{ user, setUser }}>
      {children}
    </AuthContext.Provider>
  );
};

```

Các file React component:

- Login.jsx: Nút đăng nhập Google, chuyển hướng /auth/google
- Dashboard.jsx: Hiển thị thông tin người dùng, nút logout (/auth/logout)
- Sử dụng AuthContext để kiểm tra trạng thái đăng nhập trên App.

Bước 8: Liên kết frontend & backend

- Chạy backend: node backend/server.js
- Chạy frontend: cd frontend && npm start
- Đảm bảo trong axios/fetch frontend luôn có { withCredentials: true } khi gọi API backend.
- CORS backend để origin: http://localhost:3000, credentials: true

Bước 9: Test & debug

- Đăng nhập bằng Google.
- Kiểm tra dữ liệu user trong MongoDB (đã mã hóa).
- Kiểm tra session/cookie trên trình duyệt.
- Kiểm tra middleware bảo vệ route.

7. Checklist hoàn thành

- Đăng nhập Google thành công
- Dữ liệu mã hóa trước khi lưu DB
- Phiên đăng nhập duy trì qua cookies/sessions
- Giao diện React hiển thị dashboard khi đăng nhập
- Middleware chặn truy cập nếu chưa đăng nhập

8. Testcase mẫu

1. **Đăng nhập Google:**
 - Vào trang login, nhấn nút Google, xác thực thành công, chuyển về dashboard.
2. **Kiểm tra mã hóa DB:**
 - Truy cập MongoDB, kiểm tra trường email, name đã mã hóa (không đọc được trực tiếp).
3. **Tự động đăng xuất:**
 - Không thao tác > 30 phút, reload lại, hệ thống tự logout.
4. **Chặn truy cập dashboard nếu chưa login:**
 - Đăng xuất, truy cập /dashboard, bị chuyển hướng về /login.
5. **Đăng xuất thành công:**
 - Nhấn logout, session/cookie bị xóa, không thể truy cập dashboard.

9. Lưu ý lỗi & tips, mẹo xử lý

- **AES-256:**
 - Key phải 32 bytes (64 ký tự hex), IV 16 bytes (32 ký tự hex).
- **CORS:**
 - Phải cấu hình credentials: true cả backend & frontend.
- **Session:**
 - Chỉ dùng session lưu thông tin không nhạy cảm (id user).
- **Mã hóa field:**
 - Không nên double-encrypt, kiểm tra dữ liệu trước khi mã hóa.
- **.env:**
 - Không commit file .env, để vào .gitignore.
- **Google OAuth:**
 - Đảm bảo redirect URI đúng (trùng với khai báo trên Google Console).
- **Bảo mật:**
 - Không hiển thị thông tin nhạy cảm ra frontend.
- **Debug:**
 - Dùng morgan log request, kiểm tra lỗi bằng console/error.

Chạy ứng dụng

sh

```
cd frontend
npm start
```


Tổng kết

- ✓ Xác thực Google OAuth với Passport.js
- ✓ Mã hóa dữ liệu với AES-256
- ✓ Quản lý Sessions & Cookies
- ✓ Xây dựng giao diện React.js

RÈN LUYỆN

```
[
// Dữ liệu cho Bài tập 1: User mẫu
{
  "_id": { "$oid": "666fa1f00b1e0c0012345671" },
  "name": "Alice Nguyen",
  "email": "alice@example.com"
},
{
  "_id": { "$oid": "666fa1f00b1e0c0012345672" },
  "name": "Bob Tran",
  "email": "bob@example.com"
},
{
  "_id": { "$oid": "666fa1f00b1e0c0012345673" },
  "name": "Charlie Pham",
  "email": "charlie@example.com"
},
// Dữ liệu cho Bài tập 2: User Google OAuth
{
  "_id": { "$oid": "666fa1f00b1e0c0012345674" },
  "googleId": "google-1234567890",
  "name": "David Vu",
  "email": "davidvu@gmail.com"
},
// Dữ liệu cho Bài tập 3: User đã mã hóa AES (giả lập, chỉ ví dụ)
{
  "_id": { "$oid": "666fa1f00b1e0c0012345675" },
  "name": "5e884898da28047151d0e56f8dc62927", // Giả lập chuỗi mã hóa
  "email": "8d969eef6ecad3c29a3a629280e686cf" // Giả lập chuỗi mã hóa
},
// Dữ liệu cho Bài tập 4: User có session (giả lập, không lưu trong Mongo nhưng để minh họa)
{
  "_id": { "$oid": "666fa1f00b1e0c0012345676" },
  "googleId": "google-0987654321",
  "name": "Eva Le",
  "email": "evale@gmail.com"
  // Session sẽ lưu ở Redis hoặc Memory, không lưu ở MongoDB
},
// Dữ liệu cho Bài tập 5: User frontend
{
  "_id": { "$oid": "666fa1f00b1e0c0012345677" },
  "googleId": "google-1122334455",
  "name": "Front End User",
  "email": "frontenduser@gmail.com"
}
]
```

Lưu ý sử dụng:

- Để import vào MongoDB cho các bài tập, hãy lưu file này thành `mongodb_sample_data.json` và dùng lệnh:

```
mongoimport --db secureauth --collection users --file mongodb_sample_data.json --jsonArray
```
- Các trường `name`, `email` trong bài tập 3 là chuỗi giả lập mã hóa (bạn hãy thay bằng chuỗi thực tế nếu ứng dụng mã hóa đúng).
- Nếu bạn làm từng bài nhỏ riêng biệt, có thể lọc hoặc copy từng user phù hợp vào DB của từng bài.

Bài tập 1: Hello MongoDB & Express

Mục tiêu

- Làm quen với Express, kết nối MongoDB.
- Tạo API đơn giản trả về danh sách user.

Chức năng

- Tạo server Express.
- Kết nối MongoDB.
- API GET /users: trả về danh sách user (dữ liệu mẫu).

Công nghệ sử dụng

- Node.js, Express.js
- MongoDB, Mongoose

Cấu trúc file/thư mục

```
/bai1
├── server.js
├── /models
│   └── User.js
└── package.json
```

Tóm tắt các bước thực hiện

1. Khởi tạo project, cài đặt Express, Mongoose.
2. Tạo model User.
3. Kết nối MongoDB.
4. Tạo route GET /users.
5. Chạy thử và kiểm tra API.

Test case

1. Truy cập /users trả về mảng user (dữ liệu mẫu).
2. Khi MongoDB không kết nối, trả lỗi rõ ràng.
3. Thêm user mẫu vào DB, API trả về đúng dữ liệu mới.

Bài tập 2: Đăng nhập Google OAuth cơ bản

Mục tiêu

- Thực hành xác thực Google OAuth với Passport.js.

Chức năng

- Đăng nhập bằng Google.
- Lưu thông tin user Google vào MongoDB.

Công nghệ sử dụng

- Node.js, Express.js
- Passport.js, passport-google-oauth20
- MongoDB, Mongoose

Cấu trúc file/thư mục

```
/bai2
├── server.js
├── /models
│   └── User.js
├── /config
│   └── passport.js
├── /routes
│   └── auth.js
├── package.json
└── .env
```

Tóm tắt các bước thực hiện

- Cài đặt các package cần thiết.
- Thiết lập Google OAuth trên Google Cloud Console.
- Khai báo Passport.js, route xác thực.
- Lưu user đăng nhập vào DB.
- Test đăng nhập Google.

Test case

- Đăng nhập Google thành công, user được lưu vào DB.
- Đăng nhập lại không tạo user mới trùng GoogleID.
- Nếu không xác thực thành công, trả về lỗi.

Bài tập 3: Mã hóa dữ liệu người dùng với AES-256

Mục tiêu

- Hiểu và áp dụng mã hóa dữ liệu với crypto của Node.js.

Chức năng

- Mã hóa email và tên user trước khi lưu vào DB.
- API trả thông tin user đã được giải mã.

Công nghệ sử dụng

- Node.js, Express.js
- MongoDB, Mongoose
- Crypto (Node.js built-in)

Cấu trúc file/thư mục

```
/bai3
├── server.js
├── /models
│   └── User.js
├── /utils
│   └── crypto.js
├── package.json
└── .env
```

Tóm tắt các bước thực hiện

1. Cài đặt project, kết nối DB.
2. Tạo hàm encrypt/decrypt với AES-256.
3. Model User tự động mã hóa trước khi save.
4. Route trả user đã giải mã.

Test case

1. Lưu user, kiểm tra DB thấy email, name là chuỗi mã hóa.
2. GET user trả về thông tin đã giải mã đúng.
3. Nếu thiếu key/iv, ứng dụng báo lỗi rõ ràng.

Bài tập 4: Xây dựng Middleware bảo vệ route

Mục tiêu

- Hiểu về session, cookie và middleware bảo vệ API.

Chức năng

- Đăng nhập (Google OAuth hoặc local).
- Bảo vệ route /dashboard chỉ cho user đã đăng nhập.

Công nghệ sử dụng

- Node.js, Express.js
- express-session, cookie-parser
- Passport.js (Google OAuth hoặc local)

Cấu trúc file/thư mục

```
/bai4
├── server.js
├── /middleware
│   └── auth.js
├── /routes
│   └── dashboard.js
├── package.json
└── .env
```

Tóm tắt các bước thực hiện

1. Cài session, cookie-parser cho Express.
2. Đăng nhập bằng Google/local.
3. Tạo middleware kiểm tra đăng nhập.
4. Route /dashboard chỉ cho user đã đăng nhập.

Test case

1. Chưa đăng nhập, truy cập /dashboard bị từ chối.
2. Đăng nhập xong, truy cập /dashboard thành công.
3. Đăng xuất, truy cập lại /dashboard bị từ chối.

Bài tập 5: Frontend React.js đơn giản kết nối backend

Mục tiêu

- Kết nối React frontend với backend bảo mật (session/cookie).

Chức năng

- Giao diện đăng nhập, dashboard.
- Lấy thông tin user từ backend, hiển thị trên frontend.
- Logout, tự động cập nhật trạng thái đăng nhập.

Công nghệ sử dụng

- React.js, React Router, Axios
- Node.js backend như các bài trên

Cấu trúc file/thư mục

Giải thích

```
/bai5
├── /backend
│   └── ... (server.js, routes, models, .env)
├── /frontend
│   ├── /src
│   │   ├── App.jsx
│   │   ├── /context
│   │   │   └── AuthContext.jsx
│   │   ├── /components
│   │   │   ├── Login.jsx
│   │   │   └── Dashboard.jsx
│   └── package.json
```

Tóm tắt các bước thực hiện

1. Tạo backend như các bài trước.
2. Tạo frontend React, cài Axios, Context API.
3. Giao diện đăng nhập, dashboard, logout.
4. Kết nối frontend-backend bằng API (có withCredentials).
5. Test trạng thái đăng nhập/đăng xuất liên tục trên giao diện.

Test case

1. Đăng nhập thành công, dashboard hiển thị user.
2. Đăng xuất, frontend chuyển về trang login.

3. Refresh trang ở dashboard, vẫn giữ được trạng thái đăng nhập nếu session còn hiệu lực